

Thesis Proposal
Efficient Verification of Remote Client Behavior

Robert A. Cochran
Department of Computer Science
University of North Carolina At Chapel Hill

April 16, 2013

Abstract

A misbehaving client in a distributed system can undermine the integrity of the larger distributed application in a number of different ways. For example, a server with a vulnerability may be compromised directly by a modified client. If a client is authoritative for state in the larger distributed application, a misbehaving client may transmit an altered version of this state throughout the distributed application. A player in a networked game might cheat by modifying the client executable or the user of a web site might disable client side JavaScript that validates user input parameters before submission to the server.

We present a technique for detecting whether network traffic from a remote client could have been generated by sanctioned software. Our method is based on constraint solving and symbolic execution and uses the client source code as a model for expected behavior. By identifying possible execution paths a remote client may have followed to generate a particular sequence of network traffic, we enable a precise verification technique that has the benefits of requiring little to no modification to the client application and is server agnostic; the only required inputs to the algorithm are the observed network traffic and the client source code. We demonstrate the effectiveness and performance characteristics of our technique using case studies of two multiplayer games. We also describe our plan to extend this framework to support more complex applications with improved modeling of environmental interactions and we sketch how we plan to push the limits of current symbolic execution performance to enable our method to operate on the critical path of serving client requests.

1 Introduction

A misbehaving client in a distributed system can undermine the integrity of the larger distributed application in a number of different ways. Such misbehavior may be the result of a malicious user attempting to either disrupt services or to gain advantage in a system. For example, a server with a vulnerability may be compromised directly by a modified client. Alternatively, if a client is authoritative for state in the larger distributed application, a misbehaving client may transmit an altered version of this state throughout the distributed application. Real world attacks might consist of a player in a networked game cheating by modifying the client executable or a web site user disabling client side JavaScript that validates user input parameters before submission to the server.

Existing techniques for verifying the correctness of client behavior in distributed applications suffer from imprecision, increased bandwidth consumption, or significant computational expense. A common approach to defend against client misbehavior is for the server to validate client messages using a model of client behavior derived from the sanctioned client software. For example, Giffin et al. [15] and Guha et al. [18] developed methods to confirm that requests are consistent with a control-flow model of the client. These approaches may admit false negatives however, compromised clients that make calls consistent with their control-flow models (but that may still manipulate application state) can escape detection, in a manner analogous to mimicry attacks on intrusion-detection systems [35, 30]. In other work, greater precision has been achieved, but with greater expense. For example, the *Ripley* system [32] replays each client on the server in order to validate the client’s requests, but this incurs the bandwidth overhead of transmitting all client-side inputs (user inputs, timer values, etc.) to the server to permit replay and the computational overhead of replaying the client on the server side.

We propose to resolve the tension between precision, bandwidth consumption and computational expense by constructing a behavior verification technique that validates legitimate client behavior (as being consistent with the sanctioned client software) sufficiently fast to keep pace with the application itself. In this proposal, we describe our work to date to accomplish behavior verification without encumbering the application with substantially more bandwidth use and without sacrificing accuracy. That is, any conclusion reached as to whether the sequence of client behaviors could, in fact, have been produced from the sanctioned client software is correct.

In our initial work [3], we developed a verification framework that omits transmitting client-side inputs, thus not incurring bandwidth overheads, but then must search for whether there exist inputs that could have produced the client messages observed at the server. The resulting computational expense renders this method of verification useful primarily in an offline fashion and only after modifying test applications to constrain the search spaces they present.

In our follow-up work [11], we demonstrated an extension to the aforementioned approach [3] with several advances to achieve further progress. The extended client-checking algorithm that we proposed retains precision while permitting better tradeoffs between bandwidth costs and computational expense in the common case of a legitimate client. First, we augment our verification algorithm to use a training phase to build baselines of client behavior to use in prioritizing paths to search during verification. This approach completes verification much more efficiently in the common case of a legitimate client. Second, we experimented with a framework that allows the client software to be instrumented, so that in its messages, the running client will convey “hints” that can aid verification. Such configurations that allow minimal additional bandwidth, complete verification even faster in the common case of a legitimate client. To definitively conclude that a sequence of client messages is impossible (the uncommon case), these extensions to our algorithm do not necessarily provide a performance benefit because all feasible paths must be exhaustively searched. As such, the current state of our algorithm is useful as a data reduction technique that prunes the client messages that must be logged for offline analysis by that (or another) technique. Clients whose messages are not verified quickly by our technique can be serviced only provisionally (e.g., with fewer privileges and/or logging to

enable undoing their effects) while their verification is completed offline.

We have evaluated our framework in the context of online games. Online games provide a useful proving ground for our techniques due to the frequent manipulation of game clients for the purposes of cheating [38, 24, 37] and due to the pressure that game developers face to minimize the bandwidth consumed by their games [27]. As such, our techniques are directly useful for cheat detection in this domain. Moreover, Hoglund and McGraw [19] argue that “games are a harbinger of software security issues to come,” suggesting that defenses against game cheats and game-related security problems will be important techniques for securing future massive distributed systems of other types.

We have planned several extensions to the current framework. First, is improved modeling of environment interactions to enable support of more complex applications. Networked multiplayer games are an excellent domain for our technique, but we believe with better application support, we can expand the reach of our framework. Second, we believe we can push the limits of current symbolic execution performance by minimizing the overhead of interpreting non-symbolic instructions and with extensions to the symbolic execution engine (*KLEE*) to utilize multi-core processors. Taken together, our improvements open the door to novel uses of behavior verification, including doing so in an online fashion alongside servicing client requests.

2 Background

Symbolic execution [22] is a way of “executing” a program while exploring all execution paths, for example to find bugs in the program. Symbolic execution works by executing the software with its initial inputs specially marked so they are allowed to be “anything” — the memory regions of the input are marked as symbolic and are not given any initial value. The program is executed step-by-step, building constraints on the symbolic variables based on the program’s operations on those variables. For example, if the program sets $a \leftarrow b + c$, where a , b , and c are all marked as symbolic, then after the operation, there will be a new logical constraint on the value of a that states that it must equal the sum of b and c . When the program conditionally branches on a symbolic value, execution forks and both program branches are followed, with the true branch forming a constraint that the symbolic value evaluates to true and the false branch forming the opposite constraint. Using this strategy, symbolic execution can follow every possible code path in the target program, building a constraint for each one that must hold on execution of that path.

Symbolic execution can help locate software bugs, for example, by providing constraints that enable a constraint solver to generate concrete inputs that cause errors to occur. Specifically, if execution reaches an error condition (or a state thought to be “impossible”), then a constraint solver can use the constraints associated with that path to solve for a concrete input value that triggers the error condition.

As we will see, the approach that we take to the behavior verification problem that we study is an application of symbolic execution. In our case, we symbolically execute the client software with client-side inputs unknown to the server marked symbolic and then determine whether the messages received from the client violate the postconditions derived from the software. Symbolic execution has a long history of study in the security and verification communities, but we believe the optimization problem we study here to be distinct from prior work, as discussed below.

The applications of symbolic execution that are most related to our own are in debugging and diagnostics. Zamfir et al. [42] developed a debugging tool that uses symbolic execution to reconstruct the likely path a program took before it crashed, from the core dump file recorded by the operating system when the crash occurred. Their technique finds a feasible path or set of paths through a program that allow the program to reach the memory and process state that the core dump file indicates. *SherLog* [40] is another error diagnosis tool that uses a log file instead of a core dump file to indicate how a program executed. *SherLog* performs path analysis (not symbolic execution per se, but a similar technique) to determine the likely

execution paths and variable values implied by a given set of log files. Similarly, symbolic execution has been used to discover the constraints for the paths through a program that reaches a vulnerability or error condition [4, 6, 8, 39]. Viewed through the lens of our work, the core dump file, log file, or error condition in these previous works is analogous to a “client message”, and these tools similarly seek to find an execution that could explain it. However, the structure of our verification task — namely successively building an execution path to explain an entire sequence of messages — and the performance demands that we seek to meet give rise to a technique which we believe to be novel.

Among applications of symbolic execution, software testing has received the most research attention. Symbolic execution can be an effective method of increasing the degree of code coverage in a testing tool by generating test cases that cover a high percentage of paths in a program. For example, *DART* [16] first concretely executes a program with an arbitrary input, recording the path constraint implied by its choice at each branch point. The path constraint is then modified by negating a clause and a satisfying assignment to the constraint is found to derive a new input that will cover a different path in the program. More recent examples of this approach, which is also called *concolic*¹ execution, include *CUTE* [29], *JPF* [34] and *Pex* [31, 1]. Our approach expands the verifier’s search for paths to explain client messages as needed, starting from an initial collection of paths, but it does so without solving for inputs to exercise a path concretely and without the goal of achieving high path coverage, per se.

Aside from the behavior verification that we study here, an orthogonal defense against client compromise is to strip clients of authoritative state. In this approach, any state that could affect the integrity of the larger distributed application is instead managed at the server, outside the reach of direct manipulation by the client. Tools such as *Swift* [10] automatically identify such important state for placement at the server. This approach, however, is known to increase the bandwidth consumed by interactive applications such as distributed games, owing to the need for every access to authoritative state to reach the server (e.g., [27, p. 112]). Another defense is to augment the client with monitoring software (e.g., [12, 25, 28, 14, 20]), but this approach begs the question of how to defend the monitoring software from compromise and, in some domains, has suffered resistance from the user community (e.g., [36]).

3 Framework for Verification of Client Behavior

As discussed previously, our goal is to develop a framework that will discern whether there was *any possible sequence* of client side events in the sanctioned client software that could have given rise to each message received at the server, given what the server knew about the client based on previous messages from the client and the messages the server sent to the client. More specifically, consider a sequence of messages $msg_1, msg_2, \dots$ that were sent or received by the client, listed in the order in which the client sent or received them; we call such a sequence a *message trace*. Because the server received or sent, respectively, each of these messages, the server knows their contents in addition to the order in which the client processed these messages.² As such, the message trace $msg_1, msg_2, \dots$ is known to the server and, so, the verifier. The verifier’s goal is to determine whether there exists a possible execution of the sanctioned client software along with some sequence of user inputs and system events that is *consistent* with the message trace $msg_1, msg_2, \dots$. If such an execution was possible, then we declare the message trace to be *legitimate*.

Our implementations are built on top of and use a modified version of *KLEE* [7], a symbolic execution engine designed to achieve high code coverage and find errors and inconsistencies in code. *KLEE* uses a combination of optimizations at various levels in the software stack: heuristics for selecting the best path, intelligent state management and constraint-level optimizations. The use of *KLEE* as a base for our

¹Concolic is a portmanteau of concrete and symbolic.

²Our previous work described an efficient method for the client to inform the server of the order in which the client processed these messages [3].

framework has many advantages, including its integration with the (LLVM [23]) infrastructure, providing a platform for support of new programming languages and architectures.

We leverage the power of symbolic execution to reason about the manner in which the client could have executed to generate a particular message trace. And to date, we have implemented and evaluated two iterations of our framework. The first technique, which we call *exhaustive* verification [3] and an optimized iteration of exhaustive verification called *optimistic* verification [11]. In the next two subsections we will outline these techniques.

3.1 Exhaustive Verification

At a high level, the exhaustive verification technique [2] works by generating postconditions for all of the paths through key event loops in the client application, and then during verification, constructing chains of postconditions in combination with the message trace we are verifying. If the complete postcondition chain is satisfiable, then the message trace could have been produced by a legitimate client. We omit the details of this technique here and instead focus on an overview of our follow up work [11] that is more relevant to the proposed extensions in Section 4. The computational expense required for this method of verification lends it to be useful primarily in an offline fashion and only after modifying test applications to constrain the search spaces they present.

3.2 Optimistic Verification

The optimistic verification technique [11] is optimized for the common case of verifying a message trace from a legitimate client. The method is designed to retain precision while permitting better a tradeoff between bandwidth costs and computational expense. We make use of a training stage to inform the verifier of past execution paths and an optional instrumented version of the client software that can convey “hints” while running that can aid verification.

3.2.1 Goals and Terminology

The goal of the verifier in optimistic verification is to find a sequence of client instructions, called an *execution prefix* and denoted Π , that begins at the client entry point and is *consistent* with the message trace $msg_1, msg_2, \dots$. More specifically, Π_n is consistent with $msg_1, msg_2, \dots, msg_n$ if the sequence of network I/O instructions (SEND and RECV instructions) in Π numbers n and matches $msg_1, msg_2, \dots, msg_n$ by type — i.e., if msg_i is a client-to-server message (respectively, server-to-client message), then the i -th I/O instruction is a SEND (respectively, a RECV) — and if the branches taken in Π were possible given the contents of $msg_1, msg_2, \dots, msg_n$. Note that there may be many prefixes Π consistent with $msg_1, msg_2, \dots$ (e.g., depending on other inputs to the client, such as user inputs or system-call return values), but if there are none, then the message trace $msg_1, msg_2, \dots$ is impossible given the sanctioned client software.

3.2.2 Training

Before we describe the core algorithm of our verification algorithm, we outline the training phase used to build baselines of client behavior to be used to prioritize which paths to search during verification. The training phase utilizes valid message traces (i.e., for which there exists an execution prefix consistent with each) that ideally reflect use of the client that is as similar as possible to the use that is expected to drive the client behavior that will be verified. Any invalid message traces will be detected as such during the training process (albeit at substantial computational expense). Gathering valid message traces for training purposes could be done by executing the sanctioned client software artificially or by recording message traces from actual client-server sessions.

As we will discuss below, during verification the verifier will attempt to find an execution prefix Π_n that is consistent with a message trace $msg_1, \dots, msg_n$ incrementally, i.e., by appending to an execution prefix Π_{n-1} that is consistent with $msg_1, \dots, msg_{n-1}$. To do so, it searches through *execution fragments* in an effort to find one that it can append to Π_{n-1} to create Π_n . The goal of the training phase, then, is to determine the order in which to search possible execution fragments.

More specifically, let an *execution fragment* be any nonempty path (i) beginning at the client entry point or an I/O instruction in the client software (SEND or RECV), (ii) ending at an I/O instruction (possibly the same one), and (iii) having no intervening I/O instructions. Training produces a set Φ of execution fragments; i.e., each $\phi \in \Phi$ is a path in the client software from one I/O instruction (or the client entry point) to another. As we will discuss below, the verifier will examine execution fragments in an order guided by Φ to extend an execution prefix Π_{n-1} to reach an execution prefix Π_n that is consistent with a message trace $msg_1, \dots, msg_n$. Ideally, Φ would include the execution fragments that are commonly exercised during execution or reasonable approximations thereof.

For brevity, we will not detail the algorithm for constructing Φ , except to note that it consists of the medoids of a clustering of execution fragments observed in the training message traces. We use edit distance as the distance measure for clustering, calculated using *basic blocks* as the “symbols” constituting each execution fragment. Once the execution fragments are clustered by edit distance, the medoid of each cluster is added to Φ . In addition, all training messages consistent with any fragment in each cluster are retained as *indicators* for that cluster (and its medoid). Here, an execution fragment is *consistent with* a message msg if the fragment ends at an appropriate I/O instruction — SEND if msg is a client-to-server message, RECV otherwise — and, in the case of a SEND, if there exists a client state from which execution of the fragment could result in msg being sent at that SEND.

3.2.3 Algorithm

Given an execution prefix Π_{n-1} consistent with $msg_1, \dots, msg_{n-1}$, the verifier can symbolically execute [22] the sanctioned client software on the path Π_{n-1} to yield the *symbolic state* σ_{n-1} of the client. This symbolic state consists of memory regions populated by symbolic values with constraints. The constraints on symbolic values are those implied by execution of the path Π_{n-1} — e.g., every branch condition involving a symbolic value will generally add another constraint on that value, perhaps in relation to other symbolic values — and by the message trace $msg_1, \dots, msg_{n-1}$. A memory region is *concrete* if it is constrained to be a single value.

The core of the verification algorithm starts from the symbolic state σ_{n-1} and uses a subset $\Phi_n \subseteq \Phi$ to guide a search for an execution fragment that can be appended to Π_{n-1} to yield Π_n that is consistent with $msg_0, \dots, msg_n$. Intuitively, Φ_n includes the execution fragments from Φ that are deemed likely to be similar to the fragment executed by the client leading up to it sending or receiving msg_n . We defer discussing the selection of Φ_n to Section 3.2.4; here we simply stipulate that each fragment in Φ_n begins at the instruction pointed to by the program counter of σ_{n-1} and ends at a SEND or RECV if msg_n is a client-to-server message or a server-to-client message, respectively. We stress that despite these constraints, appending a $\phi \in \Phi_n$ to Π_{n-1} will not necessarily yield a Π_n consistent with $msg_0, \dots, msg_n$.

Our verification algorithm executes roughly as follows. The algorithm builds a strictly binary tree of paths, each starting from the next instruction to be executed in σ_{n-1} . (Here, by “strictly” we mean that every non-leaf node has exactly two children, not one.) The root of the tree is the empty path, and the two children of a node in the tree extend the path represented by that node through the next *symbolic branch* (i.e., branch instruction involving a symbolic variable). One child represents that branch evaluating to false, and the other represents that branch evaluating to true. The algorithm succeeds in finding a fragment with which to extend Π_{n-1} to yield Π_n if, upon extending a path, it encounters an I/O instruction that can “explain” msg_n , i.e., that yields a state with constraints that do not contradict msg_n being the I/O instruction’s message.

Perhaps the central idea in our algorithm, though, is the manner in which it selects the next node of the tree to extend. For this purpose it uses the training fragments Φ_n . There are any number of approaches, but the one we have adopted selects the path to extend to be the one that minimizes the *edit distance* to some prefix of a fragment in Φ_n (and that has not already been extended or found to be inconsistent). This strategy naturally leads to first examining the fragments in Φ_n , then other fragments that are small modifications to those in Φ_n , and finally other fragments that are further from the fragments in Φ_n .

3.2.4 Configurations

We now specify how Φ_n is populated from the set Φ of medoids resulting from clustering the execution fragments witnessed during training (Section 3.2.2). We consider two possibilities for populating Φ_n .

Default The default algorithm configuration constructs Φ_n from the contents of msg_n . If the closest training message is at distance m from msg_n , for a measure of distance described below, then the algorithm computes the set M_n^α of training messages less than distance αm from msg_n , for a fixed parameter $\alpha \geq 1$. (In Section 3.2.5, we use $\alpha = 1.25$.) An execution fragment ϕ is *eligible* to be included in Φ_n if (i) ϕ is the medoid of some cluster for which there is an indicator message $msg \in M_n^\alpha$, and (ii) ϕ begins at the instruction to which the program counter in σ_{n-1} points, where σ_{n-1} is the symbolic state that will be passed to the verification algorithm along with msg_n and Φ_n . Then, Φ_n is set to include all eligible fragments up to a limit β ; if there are more than β eligible fragments, then Φ_n consists of an arbitrary subset of size β . (In Section 3.2.5, we use $\beta = 8$.)

The distance measure between messages that we use in our evaluation in Section 3.2.5 is simply byte edit distance between messages of the same directionality (i.e., between server-to-client messages or between client-to-server messages). If msg and msg_n do not have the same directionality, then we define their distance to be ∞ , so that only training messages of the same directionality as msg_n are included in M_n^α .

Hint The “hint” configuration requires that the client software has been adapted to work with the verifier to facilitate its verification. In this configuration, the client piggybacks a hint on msg_n that is a direct indication of the execution fragment it executed prior to sending msg_n . This extra hint, however, increases the bandwidth utilized by client-to-server messages, and so it is important that we minimize this cost.

Specifically, in this configuration, the client software has knowledge of the clustering used by the verifier. (For example, the server sends it this information when the client connects.) The client records its own execution path and, when sending a client-to-server message msg_n , maps its immediately preceding execution fragment to its closest cluster in the verifier’s clustering (using edit distance on execution fragments). The client then includes the index of this cluster within msg_n as a “hint” to the verifier. The server extracts the cluster index from msg_n and provides this to the verifier.

Intuitively, the medoid ϕ of the indicated cluster should be used as the sole element of the set Φ_n . For the applications we evaluate in Section 3.2.5, however, we adapt this idea slightly and interpret this cluster index within the set of all clusters whose fragments begin at that instruction and end at a `SEND`. Then, Φ_n is set to contain only the medoid of this cluster. (If the cluster index exceeds the number of clusters whose fragments begin at that instruction, or if msg_n is a server-to-client message, then the default approach above is used to create Φ_n , instead.) In this way, the cluster hint can be conveyed in exactly $\lceil \log_2 k \rceil$ extra bits on each client-to-server message, where k is the number of clusters allowed by the verifier. While sending a hint does increase bandwidth cost, it does so minimally; e.g., in our evaluations, we have consider $k = 256$ (1 byte per client-to-server message) and $k \leq 65536$ (2 bytes per client-to-server message). For the result of these evaluations see Section 3.2.5 and Section 6.2 in [11] respectively.

Despite the fact that the client sends the hint to the server, the client remains completely untrusted in this configuration. The hint it provides is simply to accelerate verification of a legitimate client, and providing

an incorrect hint does not substantially impact the verifier’s cost for declaring the client compromised.

3.2.5 Evaluation

To evaluate performance, we apply our algorithm to verify behavior of legitimate clients of two open-source games, namely *XPilot* and *TetriNET*. We limit our attention to *legitimate* clients since this is the case in which we make a contribution; i.e., our approach is designed to validate legal behavior more quickly than previous work, but confirms illegal behavior in time comparable to what our initial work [3] would achieve. We employ these games for our evaluation for numerous reasons: they are complex and so pose challenging test cases for our technique; they are open-source (and our tools require access to source code); and games is a domain that warrants behavior verification due to the invalid-command cheats to which they are often subjected [37].

XPilot *XPilot* is an open-source, multi-player, client-server game that has been developed in many revisions over more than 15 years including, e.g., a version for the iPhone that was released in July 2009. The version we used in our evaluation is *XPilot NG (XPilot Next Generation)* version 4.7.2. Its client consists of roughly 100,000 SLOC. Beyond this, the scope of symbolic execution included all needed libraries except `Xlib`, whose functions were replaced with minimal stubs, so that the game could be run without display output. Moreover, `uClibc` was used in lieu of the GNU C library.

In the game, the user causes her spaceship to navigate a two-dimensional world occupied by obstacles, objects such as power-ups that the user can collect by navigating her spaceship over them, and both fixed and mobile hostiles that can fire on her ship (some of which are ships controlled by other players). Each player’s goal is to earn the highest score. Despite its 2D graphics, the game incorporates physics simulation; e.g., ships with more fuel have greater mass and thus greater inertia.

Upon startup, the *XPilot* client reads local files that, e.g., define the world map. (Our evaluation assumes that these initialization files are available to the verifier, as they must be to the server, as well.) The *XPilot* client then enters an event loop that receives user input and server messages, processes them (including rendering suitable changes on the client’s display), and sends an update to the server. These updates can include information about the current status of the user’s ship’s shields (whether they are up or down), weapons (whether any are firing), position, orientation, acceleration, etc. Various limitations imposed by the client, such as that a client cannot both have its shields up and be firing at the same time, are obvious targets for a user to override by modifying the game client in order to cheat. Our behavior verification technique will detect such game cheats automatically.

TetriNET *TetriNET* is a multi-player version of the popular single-player *Tetris* game. In the *Tetris* game, one player controls a rectangular gameboard of squares, at the top of which a *tetromino* appears and starts to “fall” toward the bottom at a constant rate. Each tetromino is of a size to occupy four connected grid squares orthogonally and has one of seven shapes. The tetromino retains its shape and size as it falls, but the user can reorient the tetromino as it falls by pressing keys to rotate it. The user can also move the tetromino to the left or right by pressing other keys. Once the tetromino lands on top of another tetromino or the bottom of the grid, it can no longer be moved or rotated. At that point, another tetromino appears at the top of the grid and begins to fall. Whenever a full row of the gameboard is occupied by tetrominos, the row disappears (potentially fracturing any tetrominos occupying a portion of it) and all rows above the removed row are shifted downward. *TetriNET* differs from *Tetris* by *adding* an empty row to all other players’ grids when this occurs. The goal of the game is for a player to place as many tetrominos as possible before no more can enter her gameboard, and a player wins the multiplayer version by playing longer than other players.

The *TetriNET* client is structured as an event loop that processes user inputs and advances each tetromino in its fall down the gameboard. Only once a tetromino has landed in its resting place does the game client

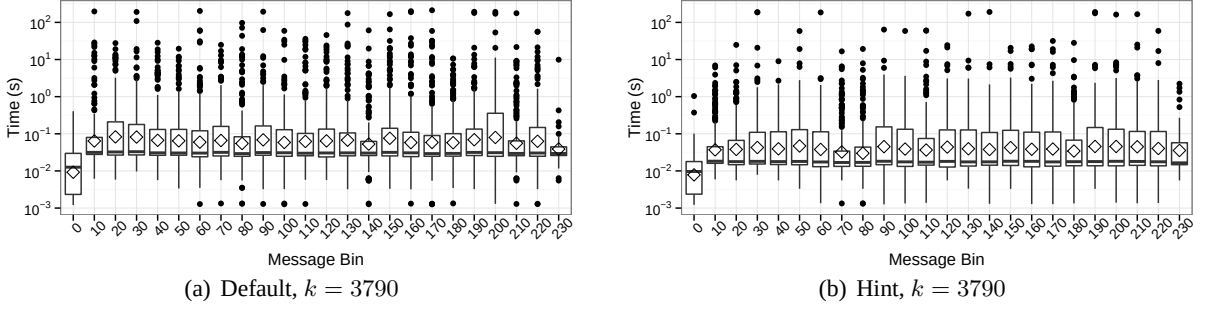


Figure 1: *TetriNET* verification times. Cross-validation over 20 traces. Boxplot at x shows verification times for messages $msg_x, \dots, msg_{x+9}$ in each trace (after training on the other traces). “ $\diamond$ ” shows the average.

inform the server of the location of the tetromino and whether its placement caused any rows to be deleted (and if so, which ones). The server does not validate the client’s claim that the condition for deleting the row was met (i.e., that the row was full), and so the game is very vulnerable to invalid-command cheats. Again, our technique will automatically detect such cheats.

The *TetriNET* client version (0.11) that we used in our evaluation is 5000 SLOC. As in *XPilot*, the scope of symbolic execution also included all needed libraries, though again the display output library (`ncurses`) was disabled using minimal stub functions and `uClibc` was used in place of the GNU C library. Despite its small size, a single event-loop iteration in the *TetriNET* client permits an unbounded number of user inputs to rotate or horizontally shift the tetromino, which presents problems for our exhaustive verification technique (Section 3.1) and required a cap of the number of inputs in a single *XPilot* event-loop iteration. Furthermore, in the evaluation of our previous technique [3], gameplay was altered so that a tetromino could be placed only at a location for which only empty squares were above it, so as to limit the number of user inputs needed for a tetromino placement to half the width of the gameboard plus the number of possible tetromino rotations — nine user inputs in total. The optimistic verification technique (Section 3.2) can operate without these restrictions. The ability of the optimistic algorithm to verify the behavior of a *TetriNET* client in its unconstrained form is one of its strengths.

3.2.6 Results

Evaluation of our verification algorithm requires traces of gameplay for both training and testing. For *TetriNET*, we generated 20 traces of manual gameplay, each of 240 messages in length (which corresponds to roughly 6.5 minutes of gameplay). For *XPilot*, we generated 40 traces of manual gameplay, each consisting of 2100 messages (roughly 70 seconds of gameplay).

Figure 1 shows *TetriNET* verification costs. Figure 1 includes plots for both the default and hint configurations, with clustering parameter value $k = 3790$; ensuring a single execution fragment per cluster. The numbers represented in Figure 1 were obtained by a 20-fold cross validation of the *TetriNET* traces; i.e., in each test, one of the traces was selected for testing, and the remainder were used for training. Specifically, Figure 1 shows the distribution of verification time per message, binned into ten-message bins, across all 20 traces. So, for example, the boxplot labeled “0” shows the distribution of verification times for messages $msg_0, \dots, msg_9$ in the 20 traces. The data point for message msg_n accounts for all time spent in the verification algorithm. In each boxplot, the “box” shows the first, second (median) and third quartiles, and its whiskers extend to ± 1.5 times the interquartile range. Additional outlier points are shown as bullets. Overlaid on each boxplot is a diamond ($\diamond$) that shows the average of the data points.

Several things are worth noting about Figure 1. In all cases, the distribution of verification times is

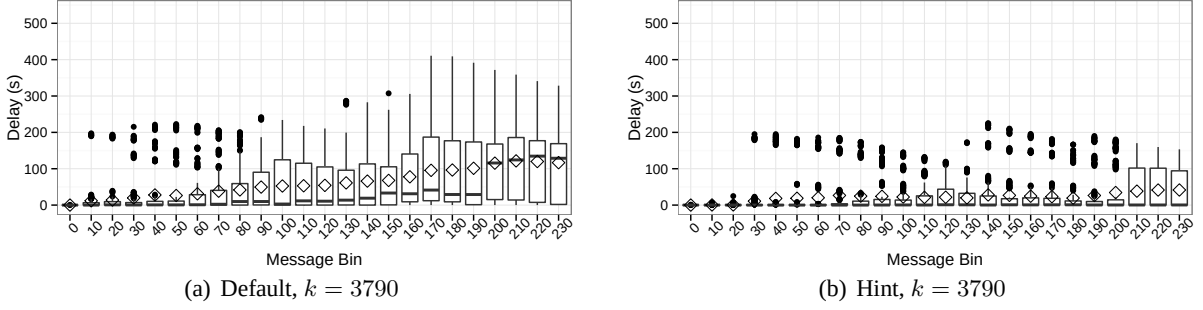


Figure 2: *TetriNET* verification delays. Cross-validation over 20 traces. Boxplot at x shows verification delays for messages $msg_x, \dots, msg_{x+9}$ in each trace (after training on the other traces). “ $\diamond$ ” shows the average.

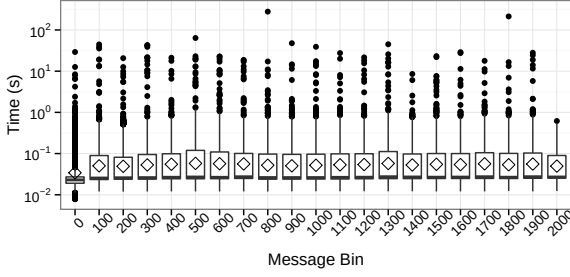
largely unaffected by the message index, i.e., where in the trace the message appears. This confirms that our implementation is mostly free from sources of increasing verification expense as traces grow longer. This figure shows the results for the fine-grained clustering, $k = 3790$, which outperforms the coarse-grained clustering (not shown, see Section 6.2 in [11]). The fine-grained hint configuration implies an overhead of 12 bits or, if sent as two bytes, an average of 17% bandwidth increase per client-to-server message. Not surprisingly, the hint configuration generally outperforms the default.

Figure 1 also suggests that our algorithm is, for the large majority of messages, fast enough to verify valid *TetriNET* gameplay at a pace faster than the game itself: the average verification cost per message, regardless of configuration or clustering granularity, is easily beneath the inter-message delay of roughly 1.6s. That said, there are two issues that require further exploration. First, there are messages that induce verification times in excess of 10s or even 100s, which unfortunately makes it impossible to reliably keep pace with gameplay. Nevertheless, as an optimization over previous work for verifying message traces, and as a data reduction technique to eliminate some traces (or portions thereof) from the need to log and analyze offline, our technique still holds considerable promise. Second, and working in favor of this promise, is the slack time between the arrival of messages that gives verification the opportunity to catch up to the pace of gameplay after a particularly difficult message verification.

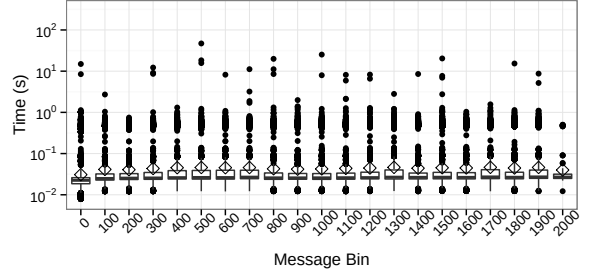
To shed light on these issues, Figure 2 instead plots the distributions of per-message verification *delay* between the arrival of message msg_n at the server (where a server-to-client message “arrives” when it is sent) and the discovery of an execution prefix Π_n that is consistent with $msg_0, \dots, msg_n$. Delay (Figure 2) differs from verification time (Figure 1) by representing the fact that verification for msg_n cannot begin until after that for msg_{n-1} completes. So, for example, the rightmost boxplot in each graph provides insight into how long after the completion of the message trace (in real time) that it took for verification for the whole trace to complete.

One item to note about these graphs is that for the hint configuration with $k = 3790$ (Figure 2(b)), the median of the rightmost boxplot is virtually zero — i.e., the most common case is that verification kept pace with gameplay. This can occur even if verification falls behind at some point in the game, since verification commonly “catches up” after falling behind. This is illustrated, for example, in the generally downward slope of consecutive outlier points in Figure 2(b). That said, the cumulative effect of verification delays in the other configurations is more costly, e.g., causing verification to lag behind gameplay by more than 100 seconds by the end of a 240-message trace in the median case in the default configuration (Figure 2(a)).

XPilot poses a significant challenge for verification because its pace is so fast. The tests described here use an *XPilot* configuration that resulted in an average of 32 messages per second. The verification times per message vary somewhat less for *XPilot* than they did for *TetriNET*, as shown in Figure 3. Recall that

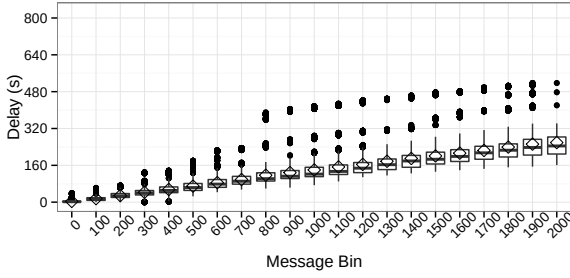


(a) Default, $k = 475$

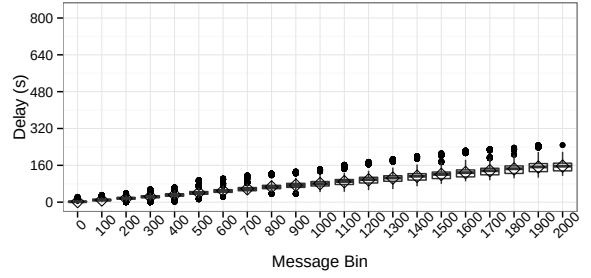


(b) Hint, $k = 475$

Figure 3: *XPilot* verification times. Cross-validation over 40 traces. Boxplot at x shows verification times for messages $msg_x, \dots, msg_{x+99}$ in each trace (after training on the other traces). “ $\diamond$ ” shows the average.



(a) Default, $k = 475$



(b) Hint, $k = 475$

Figure 4: *XPilot* verification delays. Cross-validation over 40 traces. Boxplot at x shows verification delays for messages $msg_x, \dots, msg_{x+99}$ in each trace (after training on the other traces). “ $\diamond$ ” shows the average.

each boxplot in Figure 3 represents 100×40 points, versus only 10×20 in Figure 1. As such, though there are larger numbers of outliers in Figure 3, they constitute a smaller fraction of the data points.

The median per-message verification cost of *XPilot* when clustering is fine-grained ($k = 475$, which implied a single execution fragment per cluster) is quite comparable to that in *TetriNET*, as can be seen by comparing Figure 3(a) and Figure 3(b) to Figure 1(a) and Figure 1(b), respectively. In the hint configuration, $k = 475$ increases bandwidth use by *XPilot* client-to-server messages by 4%. Though the median per-message verification cost of *XPilot* is generally as good or better than that for *TetriNET*, the faster pace of *XPilot* makes it much more difficult for verification to keep pace with the game. This effect is shown in Figure 4. As shown in this figure, none of the configurations or clustering granularities permitted verification to keep up with gameplay, and the best default configuration ($k = 475$) included one run that required 8 minutes past the end of the trace to complete its verification (see Figure 4(a)). Consequently, for an application as fast-paced and as complex as *XPilot*, our algorithm does not eliminate the need to save traces for post hoc analysis.

Nevertheless, we stress that our algorithm accomplishes, even if with some delay what was completely intractable in our previous technique [3]. That is, recall that our exhaustive verification technique (Section 3.1) utilized a restricted version of *XPilot* in which the number of user inputs per event loop iteration was artificially limited; we have removed that limitation here (see Section 3.2.5). With these restrictions removed, the exhaustive verification approach is inherently unbounded for verifying some messages, since it seeks to eagerly find *all* paths that could explain that message, of which there could be infinitely many. Our approach, in contrast, succeeds in verifying all messages in these logs in bounded time and with per-message cost averaging under 100ms in all configurations (Figure 3).

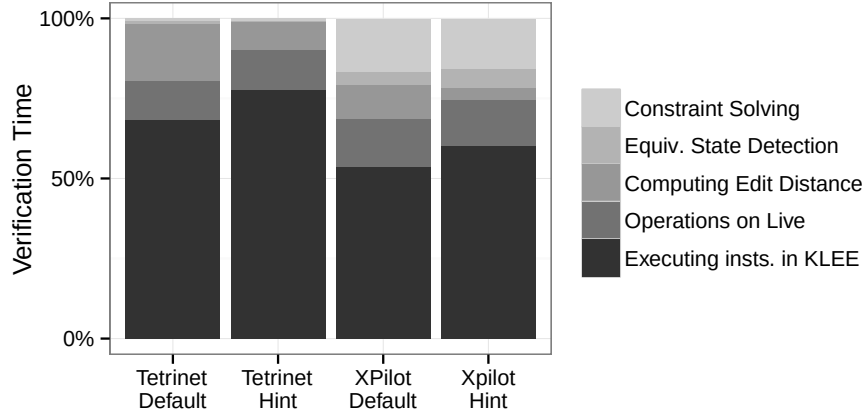


Figure 5: Percentage of time spent in each component of our algorithm.

A breakdown of verification costs for *TetriNET* is shown in Figure 5. In our *TetriNET* experiments, more than 50% of the verification time is spent in *KLEE*, interpreting client instructions. Therefore, optimizations that interpret instructions only selectively (e.g., [9]) may be a significant optimization for our tool. The majority of the remaining time is spent in operations on Live (this includes adding and removing states from a heap) and in computing edit distance, both to update the edit distance for each path when a symbolic branch is reached and to compute distances between messages. A very small fraction of time in our *TetriNET* experiments is devoted to equivalent state detection [11, Section 5.3] or in constraint solving. In Figure 5, constraint solving includes not only the time spent by *STP* (the default solver used by *KLEE*), but also preprocessing techniques to make queries to *STP* more efficient [3, Section 4.4] and a canonicalization step (borrowed from Visser et al. [33]) to improve the hit rate on cached results for previous queries to *STP*. These optimizations significantly reduce the overall constraint solving time.

A fractional breakdown of verification times for *XPilot* are also shown in Figure 5. While a majority of the cost is still contributed by interpreting client instructions in *KLEE*, the majority is smaller in the case of *XPilot* than it was for *TetriNET*. For *XPilot*, equivalent state detection [11, Section 5.3] plays a more prominent role than it did for *TetriNET*, in part due to *XPilot*'s more active use of memory allocations. Moreover, due to the substantially more complex constraints generated by *XPilot*, constraint solving plays a much more prominent role than it did for *TetriNET*.

4 Proposed Work

Our framework provides a security primitive that allows introspection into the possible paths of execution that a remote client could have taken. For our proposed research, we would like to pursue directions that both improve performance and build new functionality with this primitive.

There are two stages to the additional work. First, we plan to increase the breadth of our techniques by improving our current toolset and enabling additional case studies. Second, our aim is to develop extensions to the framework itself that improve performance and possibly enable novel forms of remote client analysis and verification. Our goals are to build a foundation for further analysis and push the envelope of symbolic execution performance in our domain.

4.1 Expanding The Scope of Verification

So far we have evaluated the technique on three applications. Two pre-existing open-source games, *XPilot* and *TetriNET*, and custom clone of *PacMan* [3]. Our experiments have been focused on these applications for two main reasons: limited language support in our verification framework and limited support for different application environments. First, our focus has been on open-source clients that are written in the C language and do not make extensive use of floating point operations. Secondly, for each application under test there are a number engineering hurdles that must be overcome before the verification framework can be used. These include, the accurate symbolic modeling of input/output events, implementation of stub functions for graphical and user interface elements, and modeling of network behavior. As we broaden the scope of our technique and its implementation becomes more robust, we hope that a larger set of candidate applications will be available and fewer special considerations will need to be handled.

A promising future domain for our technique is in the verification of complex web-based applications. Not only are web applications ubiquitous in business and communication, the JavaScript language itself is fast becoming the standard multi-platform delivery format. As a first step, we plan to support applications that are compiled with Emscripten [41], a C/C++-to-JavaScript compiler. Emscripten uses LLVM to compile and convert C/C++ applications into JavaScript, which can then be run in any modern web-browser. Javascript engines such as V8 [17] and SpiderMonkey [13] are high performance and efficient enough to execute compiled code with performance as fast as, if not faster, than hand-written code bases due to compiler based optimizations. By developing the foundation need to support networked applications that are designed or modified to be compatible with Emscripten, we will enable a wide variety of new use-cases to study. Two target applications that were originally written in C++ and then compiled to JavaScript via Emscripten are *BananaBread* and *Open Transport Tycoon Deluxe* (see Figure 6(d) and Figure 6(c)).

We now outline the challenges that will be encountered in applying our technique to additional case studies. Effort spent in enabling more experiments will provide a base for demonstrating the strengths and weaknesses of our technique.

4.1.1 Environment Modeling

Our verification framework depends on accurate modeling of the input libraries used by the client application. Thus, a model of these libraries must be implemented for each application that return input events that are symbolic, instead of actual concrete events. To verify *XPilot* and *TetriNET*, models were constructed for `Xlib` and `ncurses` respectively. For future case studies, an understanding of how an application handles user input events, timing events, and random number generation will need to be explored and a model of such an environment will need to be implemented. There is a trade-off between the effort of implementing a more general model of client-side interaction versus a specific implementation that only considers the environmental libraries used by a specific application. We plan to implement a model of the more standardized user input and graphics support library `SDL` to enable a wider variety of applications, without the need to write adhoc input handling and graphical stubs for each application.

As it is currently, our framework operates with a high-level understanding of the networking semantics of the client applications. For example, in *XPilot*, a network `RECV` call that returns the value 0 on a socket indicates that there is no longer any data to be read. However, in *TetriNET* such a return value indicates the client application has reached an error condition and our verifier terminates any execution path that reaches this state. We plan to improve our network modeling by developing a standard model that will work for a wider variety of application without the need for custom rule sets.

Another aspect of our plans to expand the scope of applications to which our techniques will apply is to examine applications with different client-side structures than those that we have examined so far, particularly ones that admit greater concurrency than we have considered to date (e.g., based on threaded

event handlers). Since this structure admits greater concurrency in its execution (i.e., concurrent execution of event handlers), this could impose greater computational cost for verifying client behavior due to the possible interleavings of event handler executions. While there are preexisting symbolic execution tools that handle nondeterministic events [21, 42, 5], our performance demands require research into new techniques to synthesize feasible client thread schedules. We will incorporate treatment of additional client structures into our verification approach, particularly ones that admit greater concurrency than we have considered to date (e.g., based on threaded event handlers).

4.1.2 Floating Point Instructions

Floating point operations are a common occurrence in games and other applications that perform physics simulations, graphical rendering, etc. The constraint solver we currently use, *STP*, does not support constraints involving floating-point numbers, and so in our previous work, we had to replace floating-point operations in *XPilot* with fixed-point operations. Forbidding use of floating-point numbers in clients will be not be viable in practice. To support client application that make use of FP operations we see several viable strategies, which might be used separately or all together. The first is to acknowledge we can already fully support any FP instruction with concrete operands. If the injected symbolic values through user inputs or environmental events do not intersect with any FP instructions, we do not need any additional support. However, if they do intersect, we could convert the FP operations to operations on the reals. There are pre-existing constraint solvers that support operations on the reals (e.g., *Z3* [26]) and they would need to be incorporated. We will also need to measure the imprecision that may be introduced by such a conversion, especially if it yields false negatives (i.e., failures to detect misbehavior) or false positives.

4.1.3 Investigation of JavaScript Interpreter

As we remarked earlier, many client-server applications are moving to web based implementations; via either hand coded applications or less commonly, C/C++ applications that are compiled to JavaScript. The ubiquitous web platform has many advantages, but also makes it very easy for a user to inject or alter code. We see two directions for support of these types of applications. The first is for applications compiled with a C/C++-to-Javascript compiler. The second is to compile and run a JavaScript interpreter (e.g., SpiderMonkey [13]) in our symbolic environment. We see an opportunity to apply our verification technique on live traffic, potentially uncovering client-to-server requests that would not have been feasible with the web application (HTML and JavaScript) that was sent to the server. We will investigate the feasibility of executing a C++ based JavaScript interpreter within our verification framework.

4.2 Performance Improvements

As stated earlier, one of the goals of our framework is online verification of network messages upon arrival. The key performance metric of our framework is the verification time of a given message, which is a measure of the time it takes our algorithm to find an execution prefix Π that is consistent with the current message sequence. We note that if the verification result is not needed on the critical path of the server’s response, then the impact of this measure can be mitigated by variation in intermessage arrival times. In other words, slack in the arrival time of messages gives more time to find Π . Nevertheless, improvements in efficiency and resource utilization can enable the verification of more clients at a lower cost. We see three avenues of performance and efficiency improvements in our framework. The first we have already addressed, using a historical training data to guide the selection of the most promising execution path to explore. This is by no means a solved problem and there are other promising research directions.

4.2.1 Parallelization

Recent efforts have demonstrated the feasibility and performance benefits of parallelized symbolic execution engines. Bucur et al. [5] demonstrated a parallelization approach that divides the symbolic execution task amongst multiple single-core worker nodes. While this approach is effective, we believe a multithreaded symbolic execution engine that is designed to utilize a multicore processor will better suit our performance needs for several reasons. We propose developing a shared memory approach that will better leverage opportunities to identify duplicate states, utilize state merging and utilize shared constraint solving caches. Furthermore, a distributed symbolic execution engine incurs cost relative to latency of communication between worker nodes. For testing at scale, this overhead is reasonable, but not for our unique high performance application of symbolic execution. Finally, our search heuristic achieves efficiency by having global knowledge of the progress of each search path, and a multithreaded shared memory symbolic execution engine will enable our algorithm to efficiently maintain information about the progress of all active candidate execution paths. We plan to parallelize the searcher and interpreter components of the *KLEE* symbolic execution engine to enable multiple paths to be evaluated concurrently.

4.2.2 Interpreter Overhead

The second performance improvement we propose is to reduce the overhead of processing an individual instruction. In our experiments thus far, a large percentage of instructions processed by the *KLEE* interpreter are fully concrete, i.e., they never encounter symbolic data, and thus there is significant potential for reduction of the overhead of interpreting the instructions. We note that Figure 5 shows the significant amount of time spent executing instructions as a percentage of the overall verification time. One approach is to perform preprocessing of the client application to identify code regions that will never, or very seldom be impacted by symbolic instructions. These regions could then be executed natively or in a more efficient emulated environment similar to S2E [9].

4.2.3 Instrumentation for Training and Client-side Hints

Our current training methods utilize pre-recorded instruction traces combined with client messages. Up to now, we have not detailed our methods for generating these traces. Currently, we record message traces from actual client-server sessions, $msg_0, \dots, msg_n$, and then generate an execution prefix Π_n by running our verification algorithm with an empty set of execution fragments, $\Phi = \emptyset$. This is done in an offline fashion. Recall that an execution fragment is a list of identifiers, that the basic blocks executed during a contiguous subset of execution. We propose development of a tool that instruments the client at compile time so that the native binary produced operates normally but additionally outputs a file containing a list of basic blocks executed. We similarly will explore using the instrumented client in the hint configuration experiments (Section 3.2.4).

5 Contributions

The following list outlines the contributions of the thesis that have already been completed.

- We have demonstrated two techniques that use symbolic execution to verify whether a sequence of messages from a remote client could have been generated by the sanctioned software [2].
- We have investigated the feasibility of using our technique to reduce the bandwidth consumption of a multiplayer game [3].
- We have demonstrated an effective technique to improve the path selection heuristic in our verification framework by selecting paths to search first based on execution history [11].

Based on current projections, we plan for the following contributions to also be included in the thesis.

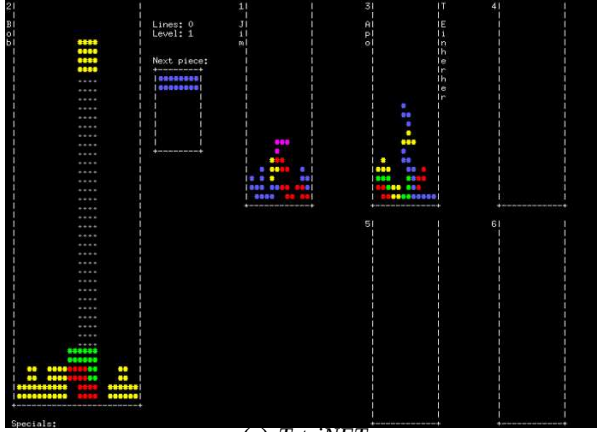
- We will develop an environmental modeling library that supports verification of applications compiled with the Emscripten C/C++-to-JavaScript compiler (Section 4.1.1).
- We will implement a solution to handle floating point instructions during verification (Section 4.1.2).
- We will parallelize core components of our framework, including the *KLEE* symbolic execution tool (Section 4.2.1).
- Continuing the initial investigation into allowing the client to provide a "hint" to the verifier [11], we will develop the necessary client instrumentation (Section 4.2.3).

Time permitting, we will explore the following additional research areas.

- We will investigate performance improvements to our framework that make use of selective non-symbolic execution of concrete functions (Section 4.2.2).
- We will investigate verification of standard web applications that, for example interact with the DOM, using our framework. As a first step, we plan to experiment with running a JavaScript interpreter within the symbolic execution framework (Section 4.1.3).

6 Anticipated Timeline

- Summer 2013
 - Initial investigation of modeling (environment and floating point) necessary for C/C++-to-JavaScript games (Section 4.1.1)
- Fall 2013
 - Implement modeling of libraries needed by C/C++-to-JavaScript applications (Section 4.1.1)
 - Develop extensions for support of floating point operations (Section 4.1.2)
 - Investigate new approaches to high-performance symbolic execution, including multi-threaded support and methods to reduce interpreter overhead (Section 4.2.2 and Section 4.2.1)
- Spring 2014
 - Experimental evaluation and fine-tuning of verification of C/C++-to-JavaScript applications (Section 4.2.2 and Section 4.2.1)
 - Initial investigation browser environment modeling to enable basic verification of web applications (Section 4.1.3)
- Summer 2014
 - Writing and paper submission
 - Thesis defense in August



(a) TetriNET



(b) XPilot



(c) Open Transport Tycoon Deluxe



(d) BananaBread

Figure 6: Screen captures of open-source multi-player games that have been evaluated with our technique (a,b) and planning to be evaluated via support for the Emscripten C/C++-to-JavaScript compiler (c,d).

References

- [1] S. Anand, P. Godefroid, and N. Tillmann. Demand-driven compositional symbolic execution. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008*, volume 4963 of *Lecture Notes in Computer Science*, pages 367–381. Mar. 2008.
- [2] D. Bethea, R. A. Cochran, and M. K. Reiter. Server-side verification of client behavior in online games. In *17th ISOC Network and Distributed System Security Symposium*, pages 21–36, Feb. 2010.
- [3] D. Bethea, R. A. Cochran, and M. K. Reiter. Server-side verification of client behavior in online games. *ACM Transactions on Information and System Security*, 14(4), Dec. 2011.
- [4] D. Brumley, J. Newsome, D. Song, H. Wang, and S. Jha. Towards automatic generation of vulnerability-based signatures. In *IEEE Symposium on Security and Privacy*, May 2006.
- [5] S. Bucur, V. Ureche, C. Zamfir, and G. Candea. Parallel symbolic execution for automated real-world software testing. In *Proceedings of the sixth conference on Computer Systems*, pages 183–198. ACM, 2011.
- [6] J. Caballero, Z. Liang, P. Poosankam, and D. Song. Towards generating high coverage vulnerability-based signatures with protocol-level constraint-guided exploration. In *Recent Advances in Intrusion Detection, 12th International Symposium, RAID 2009*, volume 5758 of *Lecture Notes in Computer Science*, pages 161–181. 2009.
- [7] C. Cadar, D. Dunbar, and D. Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *8th USENIX Symposium on Operating Systems Design and Implementation*, Dec. 2008.
- [8] C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler. EXE: Automatically generating inputs of death. In *13th ACM Conference on Computer and Communications Security*, Nov. 2006.
- [9] V. Chipounov, V. Kuznetsov, and G. Candea. S2E: a platform for in-vivo multi-path analysis of software systems. In *16th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–278, 2011.
- [10] S. Chong, J. Liu, A. C. Myers, X. Qi, N. Vikram, L. Zheng, and X. Zheng. Secure web applications via automatic partitioning. In *21st ACM Symposium on Operating Systems Principles*, pages 31–44, Oct. 2007.
- [11] R. A. Cochran and M. K. Reiter. Toward online verification of client behavior in distributed applications. In *Proceedings of the 20th ISOC Network and Distributed System Security Symposium*, 2013.
- [12] M. DeLap, B. Knutsson, H. Lu, O. Sokolsky, U. Sammapun, I. Lee, and C. Tsarouchis. Is runtime verification applicable to cheat detection? In *3rd ACM SIGCOMM Workshop on Network and System Support for Games*, Aug. 2004.
- [13] B. Eich. Spidermonkey. <https://developer.mozilla.org/en-US/docs/SpiderMonkey>.
- [14] W. Feng, E. Kaiser, and T. Schluessler. Stealth measurements for cheat detection in on-line games. In *7th ACM Workshop on Network and System Support for Games*, pages 15–20, Oct. 2008.

- [15] J. Giffin, S. Jha, and B. P. Miller. Detecting manipulated remote call streams. In *11th USENIX Security Symposium*, Aug. 2002.
- [16] P. Godefroid, N. Klarlund, and K. Sen. DART: Directed automated random testing. In *2005 ACM Conference on Programming Language Design and Implementation*, pages 213–223, June 2005.
- [17] I. Google. The v8 javascript engine. <http://code.google.com/p/v8>.
- [18] A. Guha, S. Krishnamurthi, and T. Jim. Using static analysis for Ajax intrusion detection. In *18th International World Wide Web Conference*, pages 561–570, Apr. 2009.
- [19] G. Hoglund and G. McGraw. *Exploiting Online Games: Cheating Massively Distributed Systems*. Addison-Wesley Professional, 2007.
- [20] E. Kaiser, W. Feng, and T. Schluessler. Fides: Remote anomaly-based cheat detection using client emulation. In *16th ACM Conference on Computer and Communications Security*, Nov. 2009.
- [21] S. Khurshid, C. Păsăreanu, and W. Visser. Generalized symbolic execution for model checking and testing. In *Tools and Algorithms for the Construction and Analysis of Systems, 9th International Conference, TACAS 2003*, volume 2619 of *Lecture Notes in Computer Science*, pages 553–568. 2003.
- [22] J. King. Symbolic execution and program testing. *Communications of the ACM*, 19:385–394, July 1976.
- [23] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *2004 International Symposium on Code Generation and Optimization*, Mar. 2004.
- [24] Y. Lyhyaoui, A. Lyhyaoui, and S. Natkin. Online games: Categorization of attacks. In *International Conference on Computer as a Tool (EUROCON)*, Nov. 2005.
- [25] C. Mönch, G. Grimen, and R. Midtstraum. Protecting online games against cheating. In *5th ACM Workshop on Network and System Support for Games*, Oct. 2006.
- [26] L. D. Moura and N. Bjørner. Z3: an efficient SMT solver. In *Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer-Verlag, 2008.
- [27] J. Mulligan and B. Patrovsky. *Developing Online Games: An Insider’s Guide*. New Riders Publishing, 2003.
- [28] T. Schluessler, S. Goglin, and E. Johnson. Is a bot at the controls? Detecting input data attacks. In *6th ACM Workshop on Network and System Support for Games*, pages 1–6, Sept. 2007.
- [29] K. Sen, D. Marinov, and G. Agha. CUTE: a concolic unit testing engine for C. *SIGSOFT Software Engineering Notes*, 30:263–272, Sept. 2005.
- [30] K. Tan, J. McHugh, and K. Killourhy. Hiding intrusions: From the abnormal to the normal and beyond. In *Information Hiding, 5th International Workshop, IH 2002*, pages 1–17, 2003.
- [31] N. Tillmann and J. D. Halleux. Pex: White box test generation for .NET. In *2nd International Conference on Tests and Proofs*, pages 134–153, 2008.
- [32] K. Vikram, A. Prateek, and B. Livshits. Ripley: Automatically securing Web 2.0 applications through replicated execution. In *16th ACM Conference on Computer and Communications Security*, Nov. 2009.

- [33] W. Visser, J. Geldenhuys, and M. B. Dwyer. Green: reducing, reusing and recycling constraints in program analysis. In *20th ACM International Symposium on the Foundations of Software Engineering*, FSE, pages 58:1–11, 2012.
- [34] W. Visser, C. S. Păsăreanu, and S. Khurshid. Test input generation with Java PathFinder. *SIGSOFT Software Engineering Notes*, 29:97–107, July 2004.
- [35] D. Wagner and P. Soto. Mimicry attacks on host-based intrusion detection systems. In *9th ACM Conference on Computer and Communications Security*, Nov. 2002.
- [36] M. Ward. Warcraft game maker in spying row, Oct. 2005. <http://news.bbc.co.uk/2/hi/technology/4385050.stm>.
- [37] S. Webb and S. Soh. A survey on network game cheats and P2P solutions. *Australian Journal of Intelligent Information Processing Systems*, 9(4):34–43, 2008.
- [38] J. Yan and B. Randell. A systematic classification of cheating in online games. In *4th ACM Workshop on Network and System Support for Games*, Oct. 2005.
- [39] J. Yang, C. Sar, P. Twohey, C. Cadar, and D. Engler. Automatically generating malicious disks using symbolic execution. In *IEEE Symposium on Security and Privacy*, May 2006.
- [40] D. Yuan, H. Mai, W. Xiong, L. Tan, Y. Zhou, and S. Pasupathy. SherLog: Error diagnosis by connecting clues from run-time logs. In *15th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 143–154, Mar. 2010.
- [41] A. Zakai. Emscripten: an llvm-to-javascript compiler. In *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*, SPLASH ’11, pages 301–312, New York, NY, USA, 2011. ACM.
- [42] C. Zamfir and G. Candea. Execution synthesis: a technique for automated software debugging. In *5th European Conference on Computer Systems*, pages 321–334, Apr. 2010.